

Lab 02: Building an Active Inference Agent

Objective

Construct a complete `GenerativeModel` with A–E matrices, create an `ActiveInferenceAgent`, and run a perception-action loop against a `DiscreteEnvironment`.

Prerequisites

- Completed Lab 01 (Systems)
- Understanding of probability distributions, matrix normalization
- Access to the `active_inference` library

Part 1: Defining the A–E Matrices

Goal: Build a T-maze generative model with 4 states, 3 observations, and 3 actions.

States: center (0), left arm (1), right arm (2), cue location (3) Observations: neutral (0), reward (1), no-reward (2) Actions: stay (0), go-left (1), go-right (2)

1. Define the A-matrix (3×4) encoding: center and cue give neutral observation; left arm gives reward; right arm gives no-reward (with some noise).
2. Define the B-tensor ($4 \times 4 \times 3$) encoding the transition dynamics.
3. Define $C = [0, 3, -3]$ (prefer reward, avoid no-reward).
4. Define $D = [1, 0, 0, 0]$ (start at center).

```
import numpy as np
from active_inference.agent import GenerativeModel

# TODO: Define A, B, C, D matrices
# TODO: Create model = GenerativeModel(A=A, B=B, C=C, D=D)
# TODO: Print model
```

Response:

Write your response here

Part 2: Visualizing the Model

Goal: Use the visualization functions to inspect your model.

1. Call `plot_model_summary(model)` to see all matrices at once.
2. Call `plot_A_matrix(model)` with custom observation and state labels.
3. Call `plot_B_transition_graph(model)` to see the state transition graph.

```
from active_inference.visualization import (
    plot_model_summary, plot_A_matrix, plot_B_transition_graph
)

obs_labels = ["neutral", "reward", "no-reward"]
state_labels = ["center", "left", "right", "cue"]

# TODO: Generate visualizations
```

Response:

Write your response here

Part 3: Creating the Agent and Running the Loop

Goal: Run 10 steps of the perception-action loop.

1. Create a `DiscreteEnvironment` with the same A and B matrices, starting in state 0.
2. Create an `ActiveInferenceAgent` with $\gamma=4.0$.
3. Run 10 steps, printing the observation, beliefs, and selected action at each step.

```
from active_inference.agent import DiscreteEnvironment, ActiveInferenceAgent

# TODO: Create environment and agent
# TODO: Run perception-action loop for 10 steps
```

Response:

Write your response here

Part 4: Precision Sweep

Goal: Investigate how γ affects action selection.

1. Run the same scenario with $\gamma = 0.1, 1.0, 4.0$, and 16.0 .
2. For each γ , record which actions the agent selects over 10 steps.
3. Plot or tabulate the results and describe the trend.

Response:

Write your response here

Part 5: Analysis Questions

1. Did the agent visit the reward arm (state 1)? If not, what changes to the model would encourage it?
2. How did increasing γ change the agent's behavior? At what γ value did the agent become nearly deterministic?
3. What happens if you set $C = [0, 0, 0]$? Run the loop and compare with your original results.

Response:

Write your response here

Summary

Skill	Library Component	Status
Define A, B, C, D, E matrices with correct shapes	<code>GenerativeModel(A, B, C, D, E)</code>	
Visualize model structure	<code>plot_model_summary()</code> , <code>plot_A_matrix()</code>	
Create and run an Active Inference agent	<code>ActiveInferenceAgent(model, gamma)</code>	
Investigate precision-action tradeoff	Varying γ in <code>ActiveInferenceAgent</code>	
Understand the T-maze benchmark	4 states, 3 obs, 3 actions, reward preference	